

# Cahier des charges - Stream'int

Module PRO8455 - Équipe :Marine, Rayan et Warren - Durée de réalisation : 30 heures de cours - Version 1.0 (MVP) - Technologie principale : Python

## 1. Présentation du projet

Stream'int est une plateforme qui permet à plusieurs personnes de regarder une même vidéo en même temps, depuis des appareils différents. Un utilisateur crée une session et devient le diffuseur. Il choisit une vidéo puis partage un lien ou un code. D'autres personnes rejoignent la session en tant que participants. Le diffuseur contrôle la lecture pour tout le monde : quand il met la vidéo en pause, elle se met en pause chez tous les participants.

## 2. Problématique

Comment permettre à plusieurs utilisateurs connectés par Internet de regarder une même vidéo de manière synchronisée, tout en laissant le diffuseur contrôler la lecture, et en gardant une expérience fluide malgré les différences de connexion entre les participants ?

Le projet doit gérer la diffusion vidéo, les connexions simultanées, la communication en temps réel, la synchronisation entre utilisateurs et l'adaptation de la qualité vidéo selon le réseau. Comme le projet est limité à 30 heures pour 3 personnes, le périmètre visé est un MVP.

## 3. Objectifs

L'objectif général est de réaliser un prototype qui permette à un diffuseur et plusieurs participants de regarder une même vidéo à distance, avec un contrôle de lecture partagé en temps réel.

Concrètement, le projet doit permettre de :

- créer une session et la rejoindre avec un lien ou un code ;
- diffuser une vidéo sur Internet à plusieurs participants ;
- synchroniser les actions play, pause et déplacement dans la vidéo ;
- proposer plusieurs qualités vidéo et les adapter selon le réseau ;
- tester le fonctionnement avec plusieurs utilisateurs en même temps.

## 4. Rôles

Le diffuseur crée la session, choisit la vidéo, partage le lien ou le code, et contrôle la lecture (play, pause, avance, retour). C'est lui qui garde la main sur la vidéo.

Les participants rejoignent la session avec le code ou le lien, regardent la vidéo et reçoivent les commandes du diffuseur. Dans le MVP, ils ne contrôlent pas la lecture.

## 5. Synchronisation

C'est la fonctionnalité centrale du projet : tous les participants doivent rester au même moment de la vidéo que le diffuseur. Les actions à synchroniser sont play, pause, avance et retour. Quand le diffuseur fait une action, le serveur la transmet à tous les participants connectés.

Par exemple, si le diffuseur met pause à 18 minutes 32, le serveur envoie cette position à tous les participants, qui se placent à 18:32 et se mettent en pause aussi.

## 6. Communication en temps réel

La synchronisation demande une communication instantanée entre les clients et le serveur, sans recharger la page. La technologie prévue pour ça est WebSocket. Le diffuseur envoie ses commandes au serveur, qui les redistribue à tous les participants connectés.

## **7. Sessions et invitations**

Une session contient un identifiant, le diffuseur, la vidéo choisie, la liste des participants, la position de lecture et l'état de lecture.

À la création d'une session, un code unique ou un lien est généré, par exemple un code du type A82K91 ou un lien du type [streamint.fr/join/A82K91](https://streamint.fr/join/A82K91). Le participant l'utilise pour rejoindre la session.

## **8. Streaming et qualité vidéo**

La vidéo doit être diffusée en streaming, pas juste envoyée en téléchargement. Le système doit aussi gérer plusieurs connexions en même temps.

La vidéo est préparée en plusieurs qualités (360p, 480p, 720p) avec FFmpeg. Chaque participant reçoit la qualité adaptée à sa connexion, sans que ça change la synchronisation : tout le monde reste au même moment de la vidéo, même avec des qualités différentes.

Si la connexion d'un participant se dégrade, sa qualité baisse automatiquement pour garder une lecture fluide. Elle remonte quand la connexion s'améliore.

## **9. Interface et architecture**

L'interface du diffuseur affiche le lecteur vidéo, les commandes, le code ou le lien de session, et le nombre de participants. L'interface participant affiche le lecteur vidéo et les infos de la session. L'interface reste simple pour se concentrer sur la partie technique.

Le diffuseur et les participants se connectent à un serveur central en Python/Django via WebSocket et HTTP. Ce serveur gère les sessions, s'appuie sur une base de données, et pilote le traitement vidéo (FFmpeg) ainsi que le streaming vers les participants.

## **10. Technologies**

Langage : Python. Framework web : Django. Communication temps réel : WebSocket. Base de données : SQLite. Traitement vidéo : FFmpeg. Interface : HTML/CSS/JavaScript. Streaming : HLS ou solution équivalente. Versionnement : Git. Tests : pytest.

## **11. Gestion des utilisateurs**

Un système d'identification simple suffit pour distinguer le diffuseur des participants. Pas besoin d'un vrai système de comptes complet ; si ça prend trop de temps, on le simplifiera pour se concentrer sur l'essentiel.

## **12. Tests**

Les tests fonctionnels à faire : création de session, invitation d'un participant avec le code, lancement de la lecture, pause, reprise, avance/retour, et test avec plusieurs participants en même temps. Le résultat attendu est toujours le même : tous les participants suivent les actions du diffuseur.

On testera aussi différentes conditions réseau : réseau normal (lecture fluide), réseau dégradé (qualité réduite pour garder la lecture) et réseau amélioré (qualité qui remonte).

### **13. Critères de réussite**

Le projet est réussi si ce scénario fonctionne de bout en bout : un utilisateur crée une session, choisit une vidéo, obtient un lien, plusieurs personnes rejoignent la session, le diffuseur lance la vidéo, la met en pause, la relance, avance ou recule, et tous les participants suivent. Un participant avec un réseau faible doit voir sa qualité baisser automatiquement sans que la lecture soit coupée.

### **14. Planning des 30 heures**

Analyse et conception : 2h. Architecture, environnement et Git : 2h. Serveur et gestion des sessions : 4h. Interface diffuseur/participant : 3h. Streaming vidéo : 5h. Synchronisation WebSocket : 6h. Adaptation de la qualité : 4h. Tests et corrections : 2h. Documentation et présentation : 2h. Total : 30h.

### **15. Répartition de l'équipe**

- Membre 1 : backend et synchronisation (Django, sessions, WebSocket).
- Membre 2 : streaming et vidéo (FFmpeg, qualités vidéo, adaptation réseau).
- Membre 3 : interface (diffuseur, participant, lecteur vidéo, responsive).

Le travail sera intégré régulièrement, avec un suivi Git et un journal de bord pour noter l'avancement, les problèmes rencontrés et les décisions prises.

### **16. Hors périmètre du MVP**

Ne sont pas prévus dans le MVP : application mobile ou Smart TV native, paiement, abonnements, DRM, 4K, intelligence artificielle, téléchargement hors ligne, système social avancé, publicité, gros catalogue de vidéos.

Si le temps le permet, des évolutions pourront être ajoutées : catalogue de plusieurs vidéos, chat entre participants, historique de lecture, statistiques de session, ou plus de participants par session.